Project Proposal

Sentiment Analysis / Opinion Mining

Group Members: Abdul Rafay (23L-0993), Ibrahim Rashid (23L-0741), Abbas Haider (23L-0525)

1. Research Problem and Motivation

People post their opinions online all the time. Product reviews, tweets, restaurant ratings, comments under a video, all of it is full of opinions. There is way too much of this text for anyone to sit and read through it by hand. Sentiment analysis is basically about training a program to read a piece of text and figure out if it is positive, negative, or neutral, without a human having to check it manually.

For this project we want to build something that takes in raw text, like a product or movie review, and predicts the sentiment behind it. This is a practical problem because companies and researchers actually use this kind of thing to understand how people feel about a product or event without reading thousands of reviews one by one.

The part that makes this interesting for us is that real text is messy. People use sarcasm, they say something good and something bad in the same sentence (like "the food was great but the service was terrible"), they use slang, and sometimes they mix languages, like Roman Urdu or Hinglish. So our plan is not just to make a basic classifier, but to actually test how well newer NLP models handle these tricky cases compared to older, simpler methods.

2. Data Science / NLP Techniques to be Used

We are planning to start simple and then move to more advanced models step by step, so we can actually compare how much each method improves the results:

- Classical ML baselines first: Logistic Regression, Naive Bayes and SVM using TF-IDF features, just to have a benchmark to compare against.
- Deep learning models: an LSTM or BiLSTM using word embeddings like Word2Vec or GloVe, since these usually understand context better than the classical models.
- Fine-tuning a transformer model such as BERT or RoBERTa for sentiment classification. If our dataset ends up being in Urdu or Roman Urdu, we might use a model like UrduBERT instead.
- Trying a large language model with prompting (through Hugging Face or an API) just to see how a general-purpose model compares against one we actually fine-tuned for this task.

If we have time left near the end, we also want to do a small error analysis, basically pick out the sarcastic or mixed sentiment examples and see which model actually handled them better.

3. Datasets and Tools

These are the datasets we are looking at right now. We will pick one (or combine a couple) once we check them properly for size and class balance:

- IMDB Movie Reviews, a common benchmark dataset for sentiment tasks.
- Amazon Product Reviews, where star ratings can be mapped to sentiment labels.
- Twitter/X sentiment datasets from Kaggle or Hugging Face, which are shorter and messier and closer to real world text.
- A Roman Urdu sentiment dataset from Kaggle or GitHub, in case we go with the multilingual angle.

For tools, we plan on using Python with scikit-learn for the classical models, PyTorch and Hugging Face Transformers for the deep learning and transformer part, Pandas and NumPy for cleaning the data, and Matplotlib or Seaborn for the graphs. For training, we will use Google Colab or Kaggle Notebooks since they give free GPU access.

4. Evaluation Metrics

Since this is a classification problem, we plan to check our models using these metrics:

- Accuracy, just to get a general sense of how many predictions are correct.
- Precision, Recall and F1 score, which matter more here because sentiment datasets are usually imbalanced (a lot more positive reviews than negative ones, for example).
- A confusion matrix, so we can actually see where the model is getting confused, like mixing up neutral with negative.
- A small qualitative check where we manually look at a few tricky examples (sarcasm, mixed opinions) and compare how each model dealt with them.

All the models will be tested on the same test set so we can compare them fairly.